

Concepts

Configuration

NautilusTrader uses typed configuration structs throughout the platform. Each component (data clients, execution clients, engines, strategies) has a dedicated config struct that controls its behavior.

Design principles

Defaults resolve at the config boundary

Config structs carry concrete values for fields that always have a sensible default. Timeouts, retry counts, backoff delays, and heartbeat intervals are plain types like `u64` or `u32` with defaults baked in. Downstream code receives resolved values and does not repeat defaulting logic.

Option means semantic absence, not "use default"

`Option<T>` fields appear only when `None` carries real meaning: a feature is off, a lookback window is unbounded, or a value is inherited from the environment at runtime. If a field always resolves to a concrete value, it is not wrapped in `Option`.

This distinction makes config semantics visible in the type. A plain `u64` field always has a value. An `option<u64>` field might be absent, and the code that consumes it will branch on that.

Single source of truth for defaults

Each config struct uses `bon::Builder` to define defaults in one place via `#[builder(default = value)]` annotations. The `Default` impl delegates to the builder (`Self::builder().build()`), so there is no second copy of default values that could drift out of sync.

Config decoding fails on unknown fields

Config decoding fails fast on unknown fields. Nautilus treats extra keys as bugs, not as harmless input. This catches misspellings, stale names after config renames, and copy-paste mistakes before a node or client starts with the wrong settings.

Python configs

Python config classes (msgspec structs) accept `None` for optional parameters. For plain `T` fields, `None` means "use the default." For `Option<T>` fields, `None` preserves the field's optional meaning (disabled, unbounded, etc.).

All Python config classes inherit from `NautilusConfig`, which sets `forbid_unknown_fields=True` on the underlying `msgspec.Struct`. Unknown keys now raise `msgspec.ValidationError` during decoding.

```
from nautilus_trader.adapters.bybit.config import BybitDataClientConfig

# All defaults: 60s timeout, 3 retries, etc.
config = BybitDataClientConfig()

# Override just the timeout
config = BybitDataClientConfig(http_timeout_secs=30)

# Disable instrument status polling
config = BybitDataClientConfig(instrument_status_poll_secs=None)
```

Rust configs

All config structs derive `bon::Builder`, which generates a type-safe builder with compile-time checks for required fields. Fields with `#[builder(default = value)]` can be omitted from the builder call and will use their declared default. Three equivalent ways to construct a config:

Rust config structs that deserialize with Serde also set `#[serde(deny_unknown_fields)]`.

Unknown keys now fail deserialization instead of being ignored.

```
// Builder: only set what differs from defaults
let config = BybitDataClientConfig::builder()
    .http_timeout_secs(30)
    .build();
```

```
// Struct literal with default spread
let config = BybitDataClientConfig {
    http_timeout_secs: 30,
    ..Default::default()
};

// Full defaults
let config = BybitDataClientConfig::default();
```

All three produce identical results for unspecified fields.

Common config fields

Most adapter configs share a common set of fields:

Field	Type	Default	Purpose
<code>http_timeout_secs</code>	<code>u64</code>	60	REST request timeout.
<code>max_retries</code>	<code>u32</code>	3	Maximum retry attempts.
<code>retry_delay_initial_ms</code>	<code>u64</code>	1,000	Initial backoff delay.
<code>retry_delay_max_ms</code>	<code>u64</code>	10,000	Maximum backoff delay.
<code>heartbeat_interval_secs</code>	<code>u64</code>	varies	WebSocket keepalive interval.
<code>recv_window_ms</code>	<code>u64</code>	varies	Signed request expiry window.
<code>update_instruments_interval_mins</code>	varies	varies	Periodic instrument refresh.

Adapter-specific fields (rate limits, polling intervals, margin modes) are documented in each adapter's integration guide.

Engine configs

Engine configs (`LiveExecEngineConfig`, `DataEngineConfig`, etc.) follow the same pattern. Fields like `reconciliation`, `inflight_check_interval_ms`, and `open_check_threshold_ms` are plain types with builder defaults. Genuinely optional features use `Option<T>`:

```
from nautilus_trader.config import LiveExecEngineConfig

config = LiveExecEngineConfig(
    reconciliation=True,
    open_check_interval_secs=30.0,      # Enable open order polling
    open_check_lookback_mins=60,       # Look back 60 minutes
    # position_check_interval_secs=None # Disabled by default
)
```

[< Visualization](#)[Previous Page](#)[Live Trading >](#)[Next Page](#)